

AGV Maze Challenge

HTL Braunau — Maker-Week

Betreuer: Robert Berger, Matthias Grimmer (sowie weitere betreuende Lehrkräfte)

1. The Task

Take the AGV (Automated Guided Vehicle) you built and programmed in class and make it solve a maze **as fast as possible** — fully on its own, with no human touching anything once the race starts.

The maze is built on the floor out of blue duct tape:

- an **outer rectangular box** (the boundary of the playing field) — this will never change
- a number of **inner walls**, also blue duct tape, forming the maze — these can change at any time, without notice

Your goal: get your AGV from the **start side** to the **opposite side** without ever touching the tape, in the shortest time possible.

A fixed webcam is mounted on the ceiling, looking straight down at the maze. Your program runs on a laptop/PC, pulls the video stream from the camera, figures out where your AGV is and where the maze walls are, plans a path, and sends drive commands to the AGV over the network.

You're also welcome to mount additional sensors on your own AGV (we have a range of sensors available — ask if you want to try one). The one restriction: **nothing may be added to the maze itself**. The tape walls and corner markers are it — no extra lines, beacons, reflectors, or anything else placed on the floor or walls to help your own sensors.

2. Hard Rules

1. **No manual control during a race.** Once a race starts, nobody is allowed to touch a controller, keyboard, or the AGV itself. Your software must drive the entire run by itself, end to end.
2. **No touching tape.** Your AGV must not cross or hit any duct-tape wall while moving to the other side. If you bump or push another team's AGV into a wall or out of the box, that's allowed (see below) — but *your own* tape contact disqualifies your finish.
3. You may use whatever language/tooling you want, and "vibe coding" with an AI assistant is explicitly allowed for this challenge. See §8 for the one condition attached to that.

3. Teams & Rooms

- **Teams are assigned randomly**, not self-organized — we pair one student from 3AHELS with one student from 4AHELS per team. Because the two classes don't have exactly matching headcounts, most teams will have 2 people, with exactly one team of 3. Team lists go up Monday morning.
- **Working outside scheduled hours is optional, not required.** You're welcome to keep tinkering in your free time if you want to.
- We have two rooms available: the **KI-Labor** and the **Industry Space**. The KI-Labor has the original maze field; an identical second field (same outer-box size, same ArUco corner markers) will be taped down in the Industry Space as well during the week, so both rooms can be used for building and testing in parallel.

4. The Maze & The Camera

- Outer box and inner walls: blue duct tape on the floor. The **outer box is fixed** for the whole week — it will be in place and measurable from day 1, so build and test against it all week. The **inner walls (the actual maze layout) will be changed shortly before the contest** on Friday, so don't hardcode your maze layout or path — your software needs to detect the walls fresh from the camera and (re-)plan a path on the day, not rely on a

layout you memorized during the week.

- Four **ArUco markers** (dictionary `DICT_4X4_100`) are placed at the four outer corners of the box. Use them to detect the playing field in the camera image and to undistort/rectify it into a clean top-down view.
- Every AGV carries its **own ArUco marker** on top, so the camera can identify *which* car is which, and — from the marker's orientation — which way it's currently facing.
- The webcam streams MJPEG over HTTP. Stream URL for the KI-Labor camera:

```
http://10.250.150.224:8081/
```

The Industry Space camera stream URL will be announced separately once it's set up.

Open the stream in a browser first to confirm you can see the maze before writing any code against it.

- Corner marker IDs and which corner each one marks will be announced together with the maze layout (don't hardcode IDs from any example/demo code you might find lying around — confirm them on the day). Your own AGV's marker ID will be assigned to your team.

5. Talking to Your AGV

Each AGV runs a small HTTP API on its onboard controller — that's how you send it drive commands and check on its status. Full REST API documentation:

```
https://agv-doc.github.io/
```

The base URL is your specific AGV's IP address on the lab network — it will be on a label on your car, or discoverable on the network as set up in class. Double-check which motor is "left" and which is "right" on your specific build; that's a per-robot constant you'll need to figure out yourself (and possibly invert).

6. Firmware — Do This First

Before you write a single line of vision/navigation code, update your AGV to the latest firmware. Several improvements have been made in motor control and the HTTP API since the version most cars were built with — racing on old firmware will cost you far more time than the update takes.

Firmware repository (with installation instructions included):

```
https://git.htl-braunau.at/Robert.Berger/AGV-ESP32-Firmware
```

Do this on Monday morning, before anything else.

Bring on Monday: your AGV, charged, plus the cable you use to flash/program it. Without the cable you can't update the firmware or recover from a bad flash, so don't forget it.

7. Race Format

- **Heats are 1v1, direct knockout, one attempt.** Two AGVs start at the same time, each trying to reach the opposite side first. There's no rematch within a heat — whatever happens on that one run decides the heat.
- First AGV to fully cross to the other side, without having touched tape itself, wins the heat.
- **Contact is allowed.** You may push the other AGV — into a wall, or clean out of the box. If you do, and you still complete your own run to the other side without touching a wall yourself, you win the heat. (In other words: causing your opponent to foul doesn't help you if you then foul too.)
- **Final race: Friday, 09:00.** Single-elimination bracket, heats of two AGVs at a time, repeated round by round until one AGV remains undefeated. Bring your AGV charged and ready before 09:00 — the bracket starts on

time.

8. Vibe Coding Is Allowed — With One Condition

You're free to lean on an AI coding assistant heavily for this. The condition: **you still need to be able to explain your own code**. If you (or a teammate) can't walk through what a piece of your program does and why, during the contest or in a quick check beforehand, that's a problem — not because vibe coding is banned, but because code nobody on the team understands is code nobody on the team can debug at 08:55 on Friday when something's not working.

Practical guidelines:

- Keep functions short and named for what they do.
- Comment the *non-obvious* bits (why a threshold is what it is, what a magic number means, what coordinate frame something is in) — not the obvious bits.
- Structure your program in clear stages (e.g. "grab frame" → "find markers" → "rectify image" → "locate AGV" → "plan path" → "send drive command") so you can reason about and debug one stage at a time.
- Test each stage in isolation before chaining them together (e.g. get marker detection rock-solid and visualized before you ever try to drive the motors from it).

9. General Technical Hints

You need to figure out the actual algorithm yourselves — that's the challenge. But as pointers to the right tools:

- **OpenCV** (`opencv-python` / `opencv-contrib-python`) is the standard toolkit here: video capture from an HTTP/MJPEG stream, image processing, and the `cv2.aruco` module for marker detection.
- `cv2.aruco.ArucoDetector` (or the older `detectMarkers` function, depending on your OpenCV version) gives you each marker's corner pixel coordinates and ID per frame. From the four corner markers, you can compute a perspective transform (`cv2.getPerspectiveTransform` + `cv2.warpPerspective`) to turn the angled camera view into a clean top-down map with known real-world scale (pixels per cm).
- A marker's four corners also tell you its center (position) and the direction one edge points (heading) — that's how you track your own AGV's pose every frame.
- Detecting the tape walls is an image-processing problem of its own (think color thresholding/segmentation in HSV space, then geometry) — get this working and visualize it before trusting it for navigation.
- For getting from A to B around obstacles: think about how to represent the maze as something you can search over (e.g. a grid or graph), and a textbook pathfinding approach to find a route through it. Plenty of standard algorithms exist for "shortest path around obstacles" — you don't need to invent one.
- For actually driving: you'll be sending step counts to two wheels over HTTP with some latency, not setting a continuous velocity. Think about how you turn "I want to face direction X and I'm currently facing direction Y" into discrete left/right step commands, and how often you can realistically poll the camera + send a new command (closed-loop control with a slow-ish feedback loop).
- Build in a way to *watch* what your program thinks is going on (e.g. draw the detected corners, your AGV's estimated position/heading, and your planned path onto the rectified image) — debugging blind against a live robot is painful.

10. Schedule

There are no other scheduled breaks besides the one hour lunch — handle anything else (toilet, etc.) individually as needed and as short as possible.

Lunch: 11:35 - 12:35

Day	Lessons used	Clock time	Focus
Monday	1–4, 6–8 (7 lessons)	07:50–15:10	Firmware update (§6) — bring your AGV + flash cable. Get the camera stream and ArUco corner/AGV detection working and visualized.
Tuesday	1–4, 6–8 (7 lessons)	07:50–15:10	Maze (wall) detection + perspective rectification solid and visualized. Start path planning.
Wednesday	1–4, 6–8 (7 lessons)	07:50–15:10	Path planning working end-to-end on a static image; start closed-loop driving (AGV moves itself toward a goal).
Thursday	1–4, 6–7 (6 lessons)	07:50–14:15	Full pipeline integration: camera → maze model → path → live driving. Tune, debug, handle edge cases (lost marker, recovery, etc.).
Friday	1–4, 6 (5 lessons)	07:50–13:25	Final tuning and dry runs in lesson 1. Race at 09:00 (during lesson 2).

Budget your time accordingly — a robot that reliably crosses the maze slowly beats one that's theoretically fast but never finishes a clean run.

11. Judging

Time is measured from the start signal to your AGV fully crossing to the opposite side. A heat loss by foul (touching tape, or being pushed out) ends your run for that heat regardless of remaining time. The Friday bracket winner is the overall challenge winner.

Good luck, and update that firmware before you do anything else.